DEFENSE REPORT

Vulnerability Remediation

TicketSenpai: concert-ticketing application

Prepared by: Team Sentinel

Team members: Bibas Kandel, Bipul Adhikari, Om Nepal, Sushant Aryal, Suwan Aryal

Original pentest report by: The team that tested our application (anonymous)

Date of remediation: October 27 – November 1, 2026

Report date: November 1, 2026

Course: CSC 489: Web Application Security, Capstone Milestone 3

Table of contents

1. Executive summary
2. Vulnerabilities addressed
3. Remediation details
    - 3.1 Critical and high severity fixes
    - 3.2 Medium severity fixes
    - 3.3 Low severity fixes
    - 3.4 Additional hardening
4. Testing and verification
5. Lessons learned

1. Executive summary

This report documents the security remediation of TicketSenpai, Team Sentinel's concert-ticketing application, after the Milestone 2 penetration test. The testers documented five vulnerabilities: three Critical (SQL injection, insecure direct object reference, and a forgeable session token) and two High (stored cross-site scripting and information disclosure through debug mode). We fixed all five and confirmed that each original exploit, replayed with the report's own payload, no longer works. We also fixed plaintext password storage, which the report's impact analysis exposed through the SQL injection finding. (The pentest report's executive summary mentions four vulnerabilities, but its findings section documents five; we addressed all five.)

Severity	Found	Fixed
Critical	3	3 ✓
High	2	2 ✓
Medium	0	None reported
Low	0	None reported
Additional hardening (not a reported finding)	1	1 ✓
Total	**6**	**6**

Key security improvements

- Parameterized the only SQL query that embedded user input, and range-checked numeric ids before they reach the database.
- Rendered user reviews as escaped text, enforced order ownership on the server, and replaced predictable session tokens with random ones that rotate on every login.
- Hashed passwords with salted scrypt, turned debug mode off everywhere the server starts, and stopped returning stack traces to clients.

Current security status. Every vulnerability in the penetration test report has been remediated. The backend test suite now contains a regression test for each one, and the full suite passes (20 backend tests, 65 reference tests) with all features working.

2. Vulnerabilities addressed

#	Vulnerability	Pentest finding	Severity	Status	Fixed by
1	SQL injection in event search	Finding #1	Critical	Fixed ✓	Bibas Kandel
2	Stored XSS in event reviews	Finding #2	High	Fixed ✓	Bipul Adhikari
3	IDOR on order details	Finding #3	Critical	Fixed ✓	Om Nepal
4	Forgeable session token	Finding #4	Critical	Fixed ✓	Sushant Aryal
5	Debug mode and stack-trace disclosure	Finding #5	High	Fixed ✓	Bibas Kandel
6	Plaintext password storage	(hardening)	Hardening	Fixed ✓	Suwan Aryal

Remediation approach. We worked in severity order from the report: the three Critical findings first, then the two High findings, then the password-storage hardening. Each fix was made on its own git branch and merged through its own pull request (October 27). Each one was tested immediately with the report's payload to confirm that the exploit failed and the feature still worked. Before submission we turned our Milestone 1 tests, which had asserted that the vulnerabilities existed, into regression tests that assert each exploit is blocked. We then re-ran the full suite and replayed every payload against a fresh local instance (November 1).

3. Remediation details

3.1 Critical and high severity fixes

Vulnerability #1: SQL injection in event search (Critical)

Original finding: Pentest Finding #1 · Critical · `GET /api/events?q=` · `backend/ticketsenpai/__init__.py`, `events()`

The problem. The search built its SQL with an f-string, so the `q` value became part of the query text. The payload `') UNION SELECT … FROM users --` closed the string and appended a second query that listed every account's email, role and password.

BEFORE (vulnerable): __init__.py, lines 145–146

```
sql = f"SELECT * FROM events WHERE published=1 AND (name LIKE '%{query}%' OR artist LIKE '%{query}%') ORDER BY starts_at LIMIT 20 OFFSET {offset}"
return {'items': [dict(row) for row in g.db.execute(sql)]}
```

AFTER (secure): __init__.py, lines 149–151

```
pattern = f'%{query}%'
sql = "SELECT * FROM events WHERE published=1 AND (name LIKE ? OR artist LIKE ?) ORDER BY starts_at LIMIT 20 OFFSET ?"
return {'items': [dict(row) for row in g.db.execute(sql, (pattern, pattern, offset))]}
```

What changed. The query text is constant; the pattern and offset are bound by the SQLite driver, so input is always data and can never change the query. **Security control:** parameterized query (prepared statement).

Testing verification. The report's UNION payload now returns `200 {"items": []}`, and so does `') OR 1=1 --` ; neither leaks rows or the unpublished draft event. A normal search for "After" still returns "After Hours". Regression test: `test_v01_search_injection_is_treated_as_text`.

Vulnerability #2: Stored XSS in event reviews (High)

Original finding: Pentest Finding #2 · High · `POST /api/events/<id>/reviews`, shown on `/events/<id>` · `frontend/app/events/[id]/page.js`, line 344

The problem. Reviews were inserted into the page with `dangerouslySetInnerHTML`, so the browser parsed stored review text as HTML and `<img src=x onerror=alert('XSS')>` ran for everyone who opened the event.

BEFORE (vulnerable): page.js, line 344

```
<div dangerouslySetInnerHTML={{ __html: item.body }} />
```

AFTER (secure): page.js, line 344

```
<div className="review-body">{item.body}</div>
```

What changed. The review is rendered as a React text node, so React HTML-encodes it and the browser displays it instead of executing it; no raw-HTML sink remains in the frontend. The backend stores the text as submitted, which is safe because encoding happens at output. **Security control:** context-aware output encoding (React auto-escaping).

Testing verification. We posted the report's payload and a marker variant (`onerror="document.body.dataset.xss='verified'"`), then loaded the event page in a real browser. The payload appears as literal text (Figure 1), the page contains no injected `<img>` element, and the marker was never set. Posting and reading reviews still work.

Vulnerability #3: Insecure direct object reference on order details (Critical)

Original finding: Pentest Finding #3 · Critical · `GET /api/orders/<order_id>` · `backend/ticketsenpai/service.py`, `read_order()`; route `order()` in `__init__.py`

The problem. Order details were fetched by id alone and never checked against the requester. Order ids are sequential, so Bob could open `/tickets/1` and read Alice's name, seat and barcode.

BEFORE (vulnerable): service.py, line 73; __init__.py, line 221

```
def read_order(db, order_id):
    row = db.execute('SELECT o.*, … FROM orders o … WHERE o.id=?', (order_id,)).fetchone()
# route: return read_order(g.db, order_id)
```

AFTER (secure): service.py, lines 76–82; __init__.py, line 226

```
def read_order(db, order_id, user_id):
    row = db.execute('SELECT o.*, … FROM orders o … WHERE o.id=? AND o.user_id=?', (order_id,
user_id)).fetchone()
    if row is None:
        raise Problem('order_not_found', 'This order does not exist.', 404)
# route: return read_order(g.db, order_id, g.user['id'])
```

What changed. `read_order` requires the signed-in user's id and filters on it; every caller passes it. Another user's order gets the same 404 as a missing one, so existence is not revealed. **Security control:** object-level authorization check on the server.

Testing verification. Signed in as Bob, `GET /api/orders/1` (Alice's order) returns `404 order_not_found`, and Bob's order list is empty. Alice still gets `200` for her own order. Regression test: `test_v03_order_detail_is_owner_scoped`.

Vulnerability #4: Forgeable session token (Critical)

Original finding: Pentest Finding #4 · Critical · `session` cookie on every authenticated endpoint · `backend/ticketsenpai/__init__.py`, `login()`

The problem. Login set the session cookie to the literal string `session-<user id>`. Setting `session=session-3` by hand made a visitor the administrator without a password.

BEFORE (vulnerable): __init__.py, lines 113–114

```
token = f'session-{user["id"]}'
g.db.execute('INSERT OR REPLACE INTO sessions VALUES(?,?,?)', (token, user['id'], now() + 86400))
```

AFTER (secure): __init__.py, lines 116–118

```
g.db.execute('DELETE FROM sessions WHERE user_id=?', (user['id'],))
token = secrets.token_urlsafe(32)
g.db.execute('INSERT INTO sessions(token,user_id,expires_at) VALUES(?,?,?)', (token, user['id'], now() +
86400))
```

What changed. Tokens are 256 bits from the cryptographically secure `secrets` module and cannot be guessed; each login deletes the user's previous sessions, so the token rotates. HttpOnly/SameSite=Strict cookies, 24-hour expiry and logout deletion remain. **Security control:** cryptographically random session identifiers with rotation on login.

Testing verification. `Cookie: session=session-3` on `/api/admin/events` now returns `401 unauthenticated`, and `session-1` and `session-2` return 401 on `/api/me`. A real login issues a 43-character random token, and the previous token stops working after the next login. Regression test: `test_v04_session_tokens_are_random_and_rotate`.

Vulnerability #5: Debug mode and stack-trace disclosure (High)

Original finding: Pentest Finding #5 · High · `GET /api/events/99999999999999999999999999999` · `__init__.py`, config and `unexpected()`; `service.py`, `event_for()`; `backend/run.py`

The problem. Debug mode was on, and the catch-all error handler returned `traceback.format_exc()`. An unauthenticated request with a 29-digit event id crashed SQLite binding, and the 500 response exposed source paths, the OS username, line numbers and query structure.

BEFORE (vulnerable): __init__.py, lines 17 and 44; run.py, line 4

```
app.config.update(DEBUG=True, MAX_CONTENT_LENGTH=32_768)
return jsonify(error={'code': 'debug_error', 'message': str(error), 'traceback': traceback.format_exc()}), 500
create_app().run(host='127.0.0.1', port=5001, debug=True, use_debugger=False, use_reloader=False)
```

AFTER (secure): __init__.py, lines 18 and 43–45; service.py, lines 16–17; run.py, line 4

```
app.config.update(DEBUG=False, MAX_CONTENT_LENGTH=32_768)
app.logger.exception('Unhandled API error')   # full detail stays in the server log
return jsonify(error={'code': 'internal_error', 'message': 'Something went wrong on our end.'}), 500
create_app().run(host='127.0.0.1', port=5001, debug=False, use_debugger=False, use_reloader=False)
if not 1 <= event_id <= 9_223_372_036_854_775_807:
    raise Problem('event_not_found', 'This event does not exist.', 404)
```

What changed. Debug mode is off in the app config and in the entry point `make dev` runs. Clients get a generic 500 body while details are logged server-side, and ids outside SQLite's integer range get a 404 before reaching the database. **Security control:** secure error handling, safe configuration, and input range validation.

Testing verification. The 29-digit request now returns `404 event_not_found` with no traceback. A deliberately raised server error returns only `{"error": {"code": "internal_error", "message": "Something went wrong on our end."}}`,

and the server prints "Debug mode: off" at startup. Regression test: `test_v06_errors_do_not_disclose_internals`.

3.2 Medium severity fixes

The penetration test reported no Medium severity findings.

3.3 Low severity fixes

The penetration test reported no Low severity findings.

3.4 Additional hardening

Vulnerability #6: Plaintext password storage

Issue: Not a separate finding, but Finding #1 showed its impact: the SQL injection displayed every plaintext password. · `__init__.py`, `register()` and `login()`; `db.py`, seed data

Fix applied. Passwords are stored as salted scrypt hashes (werkzeug) and verified by hash comparison, so the plaintext is never written to the database.

BEFORE (vulnerable): __init__.py, lines 101 and 109

```
INSERT INTO users(email,name,password) VALUES(?,?,?)', (address, name, password)
SELECT id,email,name,role FROM users WHERE email=? AND password=?', (address, password)
```

AFTER (secure): __init__.py, lines 102 and 110–113; db.py, lines 47–51

```
INSERT INTO users(email,name,password) VALUES(?,?,?)', (address, name, generate_password_hash(password))
row = …SELECT id,email,name,role,password FROM users WHERE email=?', (address,)
if row is None or not check_password_hash(row['password'], password): raise Problem('invalid_credentials', …,
401)
```

Testing. Every stored password now begins `scrypt:32768:8:1$…`, and none matches a demo password. The demo logins still work and a wrong password returns 401. Regression test: `test_v05_passwords_are_stored_hashed`. A database created by the Milestone 1 build must be reset once with `make reset CONFIRM=reset`, as noted in the README.

Dependency update. The pentest report noted an install-time dependency advisory that "needs separate review." It was a critical published advisory affecting Next.js 16.4.0. We upgraded to the patched release, 16.4.3 (`frontend/package.json` and lockfile). `npm audit` now reports 0 vulnerabilities, and the first `make dev` no longer stops with a security notice.

4. Testing and verification

Methodology. For each finding we replayed the exact payload and steps from the penetration test report against a fresh local instance (`make setup`, `make dev`). We confirmed that (1) the exploit no longer works, (2) the response is appropriate and reveals nothing, and (3) the feature still works with legitimate input. The XSS fix was checked in a real browser.

Vulnerability	Original exploit (from the pentest report)	Before the fix	Result after the fix
SQL injection	`') UNION SELECT … FROM users --`	All accounts and passwords listed	`{"items": []}`, nothing leaked ✓
Stored XSS	`<img src=x onerror=alert('XSS')>`	Script ran for every viewer	Shown as plain text ✓
IDOR	Bob opens `/api/orders/1`	Alice's order and barcode shown	`404 order_not_found` ✓
Session forgery	`Cookie: session=session-3`	Admin dashboard, no password	`401 unauthenticated` ✓
Info disclosure	29-digit event id	500 with full traceback	`404`, no traceback ✓
Plaintext passwords	Read the `users` table	`alice-demo` readable	Only scrypt hashes ✓

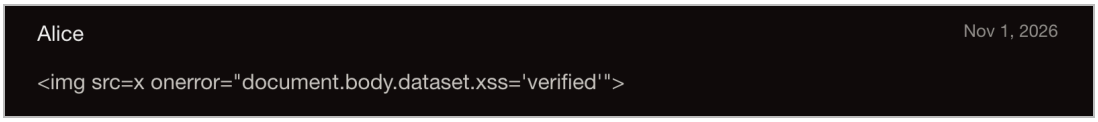


Figure 1. The stored XSS payload on `/events/1` after the fix. It shows as literal text, no image element is created, and the script does not run.

Automated regression tests. Our Milestone 1 test suite contained tests that asserted the six vulnerabilities existed. We converted them into regression tests (`test_v01` to `test_v06` in `backend/tests/test_app.py`) that assert each exploit is blocked. Run against the original Milestone 1 code, the five backend regression tests fail, as they should. The sixth, V02, is a frontend fix and was verified in the browser as described above. On the fixed code, `make test` passes all 20 backend tests and all 65 reference tests.

Functionality testing. The 14 feature tests and our manual run confirm that registration, login, logout and profile edits; event search; seat holds (including concurrency, expiry and idempotent replays); simulated checkout with decline and retry; reading one's own orders; posting and displaying reviews; and administrator event management (with 403 for customers) all still work. No feature was broken by the security changes.

5. Lessons learned

Technical lessons.

- **Each destination needs its own defense.** Data goes into SQL only through bound parameters and into HTML only through encoding. It decides whose data is returned only after an explicit ownership check.
- **Security values must be random, and secrets must be hashed.** A token derived from a user id is effectively a password everyone knows.
- **Configuration has more than one source.** We set `DEBUG=False` in the app config, but the dev-server entry point still started Flask with `debug=True`. A fix is only complete once every place the setting is applied has been checked.

Process lessons. Fixing one vulnerability per branch and testing each immediately with the report's own payload made every change small and easy to review. Our old tests encoded the vulnerabilities as expected behavior. Inverting them turned the pentest report into a permanent regression suite, so a fix can't quietly be undone.

Future improvements. Write parameterized queries and ownership checks from the first line of a feature. Keep debug output out of anything but local development. Run a security linter and static analysis during development. Review the OWASP Top 10 before building rather than after. Add login throttling, which is still missing and is our next hardening item.

Best practices adopted. Parameterized queries for all SQL; output encoding for all user content and no raw HTML sinks; server-side authorization on every user-specific endpoint; random, rotating, expiring session tokens; salted adaptive password hashing; generic client errors with server-side logging; validation of input ranges at the edge; and a regression test for every fixed vulnerability.

Conclusion. All vulnerabilities identified in the penetration test have been remediated and verified with the testers' own payloads and with automated regression tests, and the application's functionality remains intact.